Matteo LAURI - Evan STILLE - Thomas FREY - Victorien DENOYELLE

SAE 5.01

Rapport de semaine 09/10/2026

Ce qui a été fait

Cette semaine on a surtout travaillé sur un problème qui nous ralentissait depuis plusieurs semaines, installer et tester l'application était beaucoup trop compliqué. Pour que quelqu'un puisse essayer Spot'It il fallait installer Node.js pour le backend, Python avec TensorFlow pour l'IA, Flutter pour l'application et créer un compte MongoDB Atlas pour la base de données. Ensuite il fallait écrire un fichier .env à la main avec plusieurs valeurs comme l'adresse de la base, le port et la clé secrète JWT_SECRET, et la moindre erreur dedans faisait planter le serveur sans message très clair. Une fois tout installé il fallait lancer trois terminaux en même temps, un pour le backend, un pour l'IA et un pour Flutter, avec à chaque fois une commande différente à retenir.

Ça posait beaucoup d'erreurs. Certaines venaient des commandes, par exemple l'activation de l'environnement Python qui n'est pas la même dans PowerShell et dans cmd, ou TensorFlow qui ne s'installe pas avec Python 3.14. D'autres venaient du fichier .env, par exemple quand il manquait le port, quand le nom de la base n'était pas écrit dans l'adresse ou quand l'adresse IP n'était pas autorisée dans Atlas. Du coup les données de certains membres de l'équipe arrivaient dans une base appelée test au lieu de spotit. Plusieurs fois il a fallu perdre du temps à aider quelqu'un à installer le projet au lieu d'avancer sur l'application, et pour un prof ou un nouveau membre c'était presque impossible à faire seul.

Pour régler ça on a mis en place Docker. Docker permet de mettre chaque partie du projet dans une boîte appelée conteneur, avec déjà tout ce qu'il faut dedans. Comme ça on n'installe plus Node.js, Python ni Flutter sur son ordinateur, il suffit d'avoir Docker Desktop. On a créé quatre conteneurs : la base de données MongoDB, le service d'IA en Python, le serveur backend en Node.js et l'application Flutter qui est compilée en site web et servie par nginx. Un fichier docker-compose.yml les démarre tous ensemble et dans le bon ordre, c'est-à-dire la base et l'IA d'abord, puis le backend, puis l'application. Pour être sûr que chaque service est vraiment prêt avant de lancer le suivant, on a ajouté deux petites routes de vérification, /health pour l'IA et /api/health pour le backend.

Maintenant tout se lance avec une seule commande, docker compose up --build, dans un seul terminal. L'application s'ouvre ensuite dans le navigateur sur localhost. C'est beaucoup plus simple qu'avant et ça marche pareil sur tous les ordinateurs, puisque chacun utilise les mêmes versions.

Pour la base de données on a deux possibilités. Par défaut Docker lance une fausse base MongoDB locale, directement sur l'ordinateur, donc on peut tester l'application sans compte MongoDB et sans aucun fichier .env. Les comptes et les voitures créés sont gardés sur l'ordinateur mais ne sont pas partagés avec l'équipe. Si on veut se connecter à notre propre base MongoDB Atlas, il suffit de copier le fichier .env.example en .env et d'y mettre notre adresse MONGODB_URI, puis de relancer la commande. On a mis à jour les fichiers .env.example avec des commentaires qui expliquent chaque ligne, pour éviter les erreurs

qu’on avait avant. On peut aussi y changer le port de l’application avec APP_PORT, ce qui nous a servi quand le port 8080 était déjà utilisé par un autre projet.

Pour que Docker fonctionne on a fait quelques petites modifications dans le code. Dans l’application Flutter l’adresse du serveur n’était plus écrite en dur, elle se règle maintenant à la compilation avec l’option API_HOST, et par défaut ça reste http://localhost:3000 donc rien ne change pour ceux qui lancent le projet à la main. On a aussi ajouté les routes de vérification dans le backend et dans l’IA.

On a complété le README avec une grande partie « Démarrage rapide avec Docker » écrite pour quelqu’un qui part de zéro sur Windows. Elle explique comment installer Docker Desktop, comment télécharger le projet, quelles commandes lancer, comment arrêter ou tout remettre à zéro, et elle contient un tableau des erreurs les plus fréquentes avec leur solution.

Enfin on a vérifié que ça marche vraiment. On a supprimé tout ce qui avait été installé pour le projet sur l’ordinateur, c’est-à-dire Python et son environnement, les caches et les anciennes images Docker, puis on a refait l’installation comme un nouvel utilisateur en suivant seulement le README. Les quatre conteneurs sont bien passés en état healthy, l’application s’ouvre dans le navigateur et le backend affiche bien la connexion à la base.

Difficultés rencontrées

Le premier lancement de Docker est long à cause du téléchargement de TensorFlow et de Flutter. Enfin quelques dossiers de l’ancien Flutter sont difficiles à supprimer sur Windows à cause d’un nom de dossier qui commence par un espace.

Traçabilité de l’IA

Le travail de cette semaine a été fait en mode hybride nous avons fait les choix et écrit le code nous-mêmes, mais nous avons utilisé Claude pour comprendre deux sujets qu’on ne maîtrisait pas, le découpage en conteneurs Docker et la structuration d’un gros README.

“à chaque fois qu’on veut faire tester l’appli a quelqu’un faut qu’il installe node, python, tensorflow, flutter et mongodb et ca prend des heures avec plein d’erreurs différentes, comment on peut simplifier ca pour que ce soit facile pour n’importe qui”

puis, une fois la structure générale comprise, des questions plus précises : comment faire en sorte que le backend attende que MongoDB et le service IA soient prêts avant de démarrer (“healthcheck”), comment rendre l’adresse du serveur modifiable à la compilation côté Flutter plutôt qu’écrite en dur, et comment permettre d’utiliser MongoDB Atlas à la place de la base locale sans que ce soit obligatoire.

Les réponses obtenues ont servi de base pour le docker-compose.yml, les Dockerfiles et le plan du README, mais tout a été adapté et testé par nous : les noms des services, les ports, l’ordre réel des sections du README et le contenu des commandes ont été vérifiés en relançant plusieurs fois le projet en entier pour être sûrs que tout fonctionnait vraiment, et pas seulement sur le papier.

Le travail de cette semaine a été fait en mode hybride : nous avons fait les choix et écrit le code nous-mêmes, mais nous avons utilisé Claude pour comprendre deux sujets qu'on ne maîtrisait pas, le découpage en conteneurs Docker et la structuration d'un gros README.